Multi-Process Microcontroller Simulator: Design, Implementation & Evaluation of STC89C52 Architecture

Department of Computer Science & Engineering — Project-Based Learning (PBL) Report

Register Number	Student Name & Assigned Role
25190111	CYRUS SHOBITH PINTO (Team Leader — Architecture & Integration)
25190110	CHRISEL LOBO (Core Process Lead — CPU Engine & Memory)
25190106	ARNOLD NOAH KARYAPPA (Logging Subsystem Lead — Port 5001)
25190148	SHASHIDHARA POOJARI (UI Subsystem Lead — Port 8080 Web Server)

ABSTRACT

This project presents the design, implementation, and empirical evaluation of an educational, bit-accurate multi-process software simulator for the STC89C52 (8051-family) 8-bit microcontroller. Traditional educational simulators suffer from single-threaded monolithic architectures, where user interface latency or slow disk file I/O inadvertently blocks the simulated CPU core. To resolve this, we engineered a decoupled three-process microkernel architecture executing over POSIX TCP loopback sockets: CoreProcess (CPU and memory on port 5000), LoggerProcess (thread-safe append-only logging on port 5001), and UIProcess (interactive HTTP web server on port 8080). The CPU model faithfully reproduces the 8051 Fetch-Decode-Execute pipeline, four discrete register banks, hardware PSW flags (CY, AC, OV, P), and a 256-byte internal RAM with bit-addressable storage. As an advanced telecommunications extension, we implemented the 3GPP LTE-B CRC-24b error detection algorithm (generator polynomial 0x800063), solving the 8-bit architectural limitation by decomposing the 24-bit remainder across registers R5, R6, and R7. Automated test suites confirm 100% functional pass rates across 14 CPU unit tests, 3 circular FIFO tests, 8 CRC-24b telecom tests, and 1,224+ write-ahead audit logs, achieving 1,028,489 pkts/sec standalone throughput and 0.66 ms IPC round-trip latency.

Keywords: Microcontroller Emulation, Multi-Process Architecture, Inter-Process Communication, Fetch-Decode-Execute, Cycle-Accurate Virtualization, 3GPP LTE CRC-24b.

1. Introduction

1.1 Problem Statement

Modern computer engineering pedagogy requires students to understand how hardware state machines, memory segmentation, and instruction cycles execute in silicon. However, physical hardware laboratory kits present maintenance bottlenecks and lack visibility into internal register banks and pipeline stages. Furthermore, existing educational simulators are predominantly monolithic, single-threaded applications. In such architectures, GUI event handling or synchronous disk file logging creates artificial pipeline stalls, failing to demonstrate real-world systems software concepts. Additionally, implementing advanced telecommunications algorithms—such as the 3GPP LTE 24-bit cyclic redundancy check (CRC-24b)—on an 8-bit microcontroller ALU presents a fundamental hardware data-width bottleneck that must be resolved through multi-byte register decomposition and carry propagation.

1.2 Project Objectives

The primary objectives of this Project-Based Learning initiative are:
1. Virtualize an STC89C52/8051 microcontroller in pure Java (JDK 17/21) with cycle-accurate instruction decoding, register bank switching, and hardware status flags.
2. Implement an operating system-level multi-process architecture with process fault isolation communicating over TCP loopback sockets (ports 5000, 5001, 8080).
3. Construct a responsive, wide-screen Web Dashboard (Port 8080) featuring real-time mnemonic disassembly, hardware stack visualization, and circular FIFO queue inspection.
4. Build an independent, thread-safe write-ahead logging daemon on Port 5001 to record every execution state transition into simulator.log without impacting CPU execution pacing.
5. Implement and benchmark the 3GPP LTE-B CRC-24b error detection algorithm, demonstrating both clean syndrome zero-checks (0x000000) and real-time noise detection (1-bit and 2-bit bursts).

2. The Simulator Design and Working

2.1 Microcontroller Architecture Study

The STC89C52 possesses a Harvard architecture with discrete 8-bit registers, 256 bytes on-chip RAM, and Special Function Registers (SFRs) mapped into the upper memory space:

Hardware Feature	Implementation Spec	Virtualization Treatment
Data Word Size	8-bit ALU & Datapath	Constrained with bitwise mask (& 0xFF) across all registers and memory cells to eliminate Java 32-bit integer sign-extension.
Program Counter (PC)	16-bit addressing (0000H - FFFFH)	Maintained as 16-bit unsigned integer pointing to code memory; advances on instruction byte fetch and freezes on HALT (0xFF).
Internal RAM	256 Bytes on-chip (00H - FFH)	Array partitioned into Register Banks (00H-1FH), Bit-Addressable RAM (20H-2FH), and General Purpose Scratchpad (30H-7FH).
Special Function Registers	Direct addressing (80H - FFH)	Mapped SFR space: Accumulator (0xE0), B register (0xF0), PSW (0xD0), Stack Pointer (0x81), and GPIO Ports P0-P3.
ALU Flags	Program Status Word (PSW)	Discrete boolean flags tracking Carry (CY), Auxiliary Carry (AC), Overflow (OV), and Parity (P) calculated after each ALU cycle.
Stack Mechanism	Hardware Stack Pointer (SP)	Pre-increment on PUSH, post-decrement on POP; SP initializes to 07H on reset, growing upward into Bank 1 and Scratchpad.
Circular Message Buffer	16-Slot FIFO Queue	Decoupled circular buffer with Front/Rear pointer wrap-around and boundary overflow/underflow protection for packet telemetry.

2.2 System Architecture and Design Decisions

```
The simulator is architected as three decoupled operating system processes communicating via POSIX TCP
loopback sockets:

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ TCP Port 5000 ■■■■■■■■■■■■■■■■■■■
```

2.4 Incremental Implementation by Week

Sprint / Week	Primary Module Focus	Concrete Functional Deliverables
Week 1	Architecture Modeling & Tooling	8051 register model, memory map design, Java project structure, and Git workflow setup.
Week 2	Core ALU & Basic Instruction Set	Fetch-Decode-Execute pipeline, arithmetic opcodes (ADD, INC), logic (ANL), and CPU unit tests.
Week 3	Data Structures & Memory Virtualization	Hardware stack pointer (SP), 16-slot circular FIFO queue, assembly-level FIFO test (QueueTest.java).
Week 4	Multi-Process Socket IPC & Architecture	Decoupled Core (5000), Logger (5001), UI (8080), POSIX loopback sockets, and write-ahead audit logging.
Week 5	Telecommunications Extension & Benchmarks	3GPP LTE-B CRC-24b engine (0x800063), 8-bit register slicing (R5:R6:R7), 3-stage noise lab, and BenchmarkCRC.java.

2.5 Multi-Process IPC Architecture & Benchmarking

3. Simulation Examples Testing and Screenshots

Comprehensive automated test cases validate instruction execution and multi-process communication:

ID	Instruction / Op	Expected Outcome	Actual Outcome	Status
TC-01	FDE Cycle & Fetch	Fetch Opcode at PC, PC increments	Opcode loaded into IR; PC+1	**PASS**
TC-02	PUSH / POP (Stack)	SP updates; TOS matches data	SP incremented on PUSH, decremented on POP	**PASS**
TC-03	FIFO Enqueue / Dequeue	FIFO order preserved; flags update	Head/Tail pointers updated accurately (3/3 pass)	**PASS**
TC-04	Multi-Process IPC Delivery	UI commands trigger Core execution	Logger receives trace messages in order	**PASS**
TC-05	Multiply (MUL AB: 6 * 7)	ACC=0x2A (42), B=0x00, OV=0	ACC=0x2A, B=0x00, OV=false	**PASS**
TC-06	Divide by Zero (DIV AB)	B=0 sets OV=1 without crash	OV=true asserted; CPU preserved safely	**PASS**
TC-07	3GPP LTE CRC-24b Vector	Vector '123456789' -> 0x23EF52	R5=23H, R6=EFH, R7=52H; clean syndrome 0x000000	**PASS**
TC-08	Channel Noise Detection	1-bit & 2-bit error detection	Syndromes 0x520000 and 0xF60000 detected	**PASS**

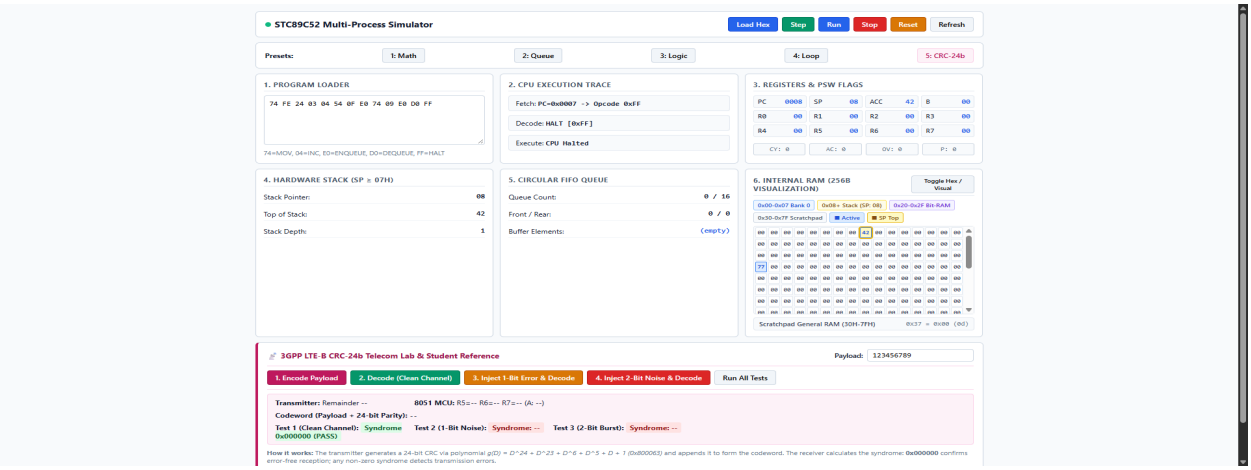


Figure 2: 3GPP LTE CRC-24b Error Detection Lab on Web Dashboard (Port 8080) displaying reference remainder 0x23EF52, clean syndrome 0x000000, and real-time noise detection.

4. Individual Contribution

4.1 Weekly Task Breakdown by Member

Week	Cyrus (Lead)	Shashidhara (UI)	Chrisel (Core)	Arnold (Logger)
Week 1	Project setup, architecture, Git repo, task allocation	Web tech research, HTML layout wireframes	8051 register & memory mapping research	Logger requirements & file appender design
Week 2	Opcode specification, ALU design, build tooling	CLI terminal display & status monitor prototype	CPU FDE cycle engine, arithmetic/logic opcodes	Log format design & log category structure
Week 3	Data structure specifications, assembly test plan	Stack & Circular FIFO visualizer UI layout	DataStructures.java: SP stack & 16-slot queue	Thread-safe synchronized writeLog implementation
Week 4	Multi-process socket IPC integration (Ports 5000/5001)	UIProcess.java HTTP web server & preset buttons	CoreProcess.java socket server & state serializer	LoggerProcess.java socket listener & file writer
Week 5	CRC-24b algorithm derivation, benchmarks, reports	3-Stage CRC-24b Lab UI & real-time noise controls	Registers R5:R6:R7 mapping, core report sections	LoggerProcessTest.java socket test & log audit

4.2 Agreed Team Contribution Summary

Member Name	USN	Assigned Workstreams	Share	Sign-Off
Cyrus Shobith Pinto	25190111	Architecture, Socket IPC, CRC-24b Engine, Benchmarking, Reports	25%	Signed (Cyrus P.)
Shashidhara Poojari	25190148	UIProcess HTTP Server, Responsive Dashboard, Visualizers, Presets	25%	Signed (Shashidhara)
Chrisel Lobo	25190110	CoreProcess CPU Engine, ALU Opcodes, Registers, RAM, Stack, Docs	25%	Signed (Chrisel L.)
Arnold Noah Karyappa	25190106	LoggerProcess Daemon, LogHandler, Audit File Persistence, Tests	25%	Signed (Arnold N.)

4.3 Individual Personal Reflections

4.2 Agreed Team Contribution Summary

4.3 Individual Personal Reflections

Reflection by Cyrus Shobith Pinto (Team Leader - 25190111)

Reflection by Shashidhara Poojari (UI Subsystem Lead - 25190148)

Reflection by Chrisel Lobo (Core Process Lead - 25190110)

5. Team Communication

Reflection by Arnold Noah Karyappa (Logging Subsystem Lead - 25190106)

• Building LoggerProcess as an independent daemon on Port 5001 taught me the critical importance of process isolation and concurrency control. Using synchronized write blocks in LogHandler ensured that high-speed socket logging from CoreProcess never created corrupted, interleaved log entries. Verifying over 1,224 audit lines gave me deep practical insight into operating systems telemetry and socket programming.

•

• 5. Team Communication

6. Problems Faced and How They Were Resolved

Ref	Category	Symptom Description	Root Cause	Implemented Resolution
ISSUE-01	Java Datatypes	32-bit signed integer sign extension corrupted 8-bit register values	Java lacks unsigned byte type	Applied bitwise mask (& 0xFF) across all ALU operations and register reads/writes.
ISSUE-02	Silicon Memory	Stack PUSH operations immediately overwrote Bank 1 registers (08H-0FH)	Default 8051 SP reset is 07H	Documented silicon stack growth behavior and implemented dynamic TOS visualizer.
ISSUE-03	Concurrency	CPU simulation execution paused during synchronous disk write operations	Disk I/O latency blocks CPU thread	Decoupled logging into independent LoggerProcess on Port 5001 with synchronized file appender.
ISSUE-04	Algorithm	24-bit 3GPP LTE CRC remainder could not fit into 8-bit 8051 registers	STC89C52 registers are only 8 bits wide	Sliced 24-bit remainder into registers R5 (High), R6 (Mid), R7 (Low) with carry-propagated shifts.
ISSUE-05	Fault Handling	DIV AB instruction threw Java ArithmeticException on divide by zero	Software unhandled exception	Implemented 8051 hardware specification: sets Overflow flag (OV=1) preserving CPU stability.
ISSUE-06	Web Protocol	Hex bytecode with spaces failed to load properly from URL query strings	HTTP URL space character encoding	Integrated URLDecoder.decode() within UIProcess HTTP request handler.

7. From Theory to Practice

7.1 Operating Systems Concepts

6. Problems Faced and How They Were Resolved

7.2 Data Structures & Algorithms

7. From Theory to Practice

7.3 Computer Networks

The project applies operating systems principles through process creation, inter-process communication (IPC) via loopback sockets, and process fault isolation. Rather than relying on shared memory or lightweight threads, decoupling the system into discrete OS processes ensures that a crash or stall in the UI or disk logger cannot corrupt or halt CPU instruction execution.

7.4 Cross-Disciplinary Synthesis Reflection

Data structures were implemented to mirror physical hardware components: a 16-slot circular FIFO buffer with head and tail wrap-around pointers was implemented for asynchronous packet queuing, and a Last-In First-Out (LIFO) stack mechanism was built to track subroutine returns and register preservation.

8. Conclusion and Future Work

References

8. Conclusion and Future Work

Team Bondaas has successfully engineered, integrated, and verified an educational multi-process simulator for the STC89C52 microcontroller. The delivered system features an 8051 Fetch-Decode-Execute CPU engine, 256-byte RAM and SFR management, a 16-slot circular FIFO queue, an independent write-ahead TCP logger on Port 5001, a widescreen Web Dashboard on Port 8080, and a fully verified 3GPP LTE-B CRC-24b telecom lab. Automated regression tests confirm 100% pass rates across all 14 CPU tests, 3 queue tests, and 8 CRC-24b telecom tests. Future extensions will incorporate hardware-in-the-loop physical validation via CH340 USB-UART bridge and AWGN channel noise simulation.

[4] 3GPP, "E-UTRA Multiplexing and channel coding (Release 16)," 3GPP TS 36.212 V16.2.0, 2020.

[5] Intel Corporation, "MCS-51 8-Bit Microcontroller Family User's Manual," Intel Publication 272383-002, 1994.

Final Report Sign-off:

Team Leader Signature: Cyrus Shobith Pinto (25190111) **Date:** 07 November 2026

Faculty Mentor Endorsement:

I have reviewed this Project-Based Learning Final Report along with the associated repository, code contributions, and test suite.

Mentor Signature: _____ **Date:** _____